UNIVERSIDAD TÉCNICA ESTATAL DE QUEVEDO

Facultad de Ciencias de la Computación — Carrera de Ingeniería de Software

FORM-FB-01: Fichas bibliográficas

Fundamentos de sistemas distribuidos y arquitecturas distribuidas

Estudiante:	Sánchez Vera Darwin Joel (dsanchezv@uteq.edu.ec)
Asignatura:	Aplicaciones Distribuidas (ISR-701) — 7.º nivel, paralelo B
Docente:	Gleiston Cicerón Guerrero Ulloa, PhD
Semana:	2 (2 al 8 de noviembre de 2026)
Modalidad:	Individual, formativa (preparatoria para TA-IND-01)

Introducción

El objetivo de estas fichas es reunir la base bibliográfica para mi informe individual TA-IND-01, en el que analizaré de forma crítica un sistema distribuido real. Busqué fuentes que me ayudaran a revisar ese sistema desde varios ángulos: cómo se despliegan y comunican sus servicios, cómo maneja los datos, cómo se mide su escalabilidad y cómo detecta sus fallos. Por eso las cinco fuentes no tratan el mismo tema, sino que se complementan.

Las cinco son artículos de revistas indexadas publicados entre 2021 y 2025, con DOI. La Tabla 1 resume sus datos principales.

Tabla 1: Fuentes seleccionadas

N.º	Revista (editorial)	Tema central	Año
1	Journal of Systems and Software (Elsevier)	Evaluación del rendimiento y la escalabilidad	2025
2	ACM Computing Surveys (ACM)	Detección de anomalías y causa raíz de fallos	2023
3	Empirical Software Engineering (Springer)	Medición de la escalabilidad en la nube	2022
4	Proceedings of the VLDB Endowment (VLDB Endowment)	Gestión de datos en microservicios	2021
5	Journal of Systems and Software (Elsevier)	Patrones de despliegue y comunicación	2021

Procedimiento de verificación

Cada fuente se verificó antes de registrarla. Primero se abrió el DOI para confirmar que dirigía a la obra citada. Después se compararon autores, año, título, revista, volumen, número y páginas (o número de artículo, en las revistas de Elsevier y Springer que no usan páginas) con el registro oficial del DOI en Crossref y con la página del artículo en el sitio del editor. Por último, los datos concretos se tomaron del texto completo de cada artículo, y se indica la sección o página donde aparecen.

Ficha 1. Rendimiento y escalabilidad de los microservicios

Referencia (IEEE): H. Rodrigues, A. Rito Silva y A. Avritzer, «Assessment of performance and its scalability in microservice architectures: Systematic literature review,» *Journal of Systems and Software*, vol. 230, 112500, dic. de 2025. DOI: 10.1016/j.jss.2025.112500

DOI verificado: 10.1016/j.jss.2025.112500

Resumen: El artículo parte de un problema práctico: aunque los microservicios se asocian con una buena escalabilidad, no existe una manera común de evaluar su rendimiento y su capacidad de crecer. Para estudiarlo, los autores realizaron una revisión sistemática de la literatura publicada entre 2014 y comienzos de 2024; revisaron 801 estudios y seleccionaron 29. Antes definieron un marco con los elementos de la arquitectura (servicios, bases de datos, middleware y despliegue) y las actividades de evaluación. Concluyen que la mayoría de trabajos se centra en el despliegue y el escalado, mientras que la gestión de datos y la comparación entre patrones casi no se estudian.

Dato concreto: El escalado, horizontal o vertical, es la preocupación arquitectónica más frecuente: aparece en 20 de los 29 estudios primarios. En cambio, solo 2 estudios consideraron una base de datos compartida y apenas uno abordó la consistencia de los datos (Sección 4.1).

Nota personal: Este artículo me hizo ver que decir «el sistema escala» no es suficiente: hay que indicar con qué carga, con qué objetivo de servicio y sobre qué recursos se mide. En TA-IND-01 usaré su marco de dimensiones como guía en el apartado de análisis de rendimiento y escalabilidad, y también para señalar qué aspectos del sistema elegido no se pueden comprobar con la información pública disponible.

Ficha 2. Detección de anomalías y análisis de causa raíz

Referencia (IEEE): J. Soldani y A. Brogi, «Anomaly Detection and Failure Root Cause Analysis in (Micro) Service-Based Cloud Applications: A Survey,» *ACM Computing Surveys*, vol. 55, n.º 3, págs. 1-39, mar. de 2023. DOI: 10.1145/3501297

DOI verificado: 10.1145/3501297

Resumen: Cuando una aplicación está formada por decenas o cientos de servicios que se llaman entre sí, detectar un fallo y encontrar su origen se vuelve difícil. Soldani y Brogi abordan este problema mediante una revisión de las técnicas disponibles para detectar anomalías y analizar la causa raíz de los fallos en aplicaciones multiservicio. Clasifican las técnicas según los datos que utilizan (registros o *logs*, trazas distribuidas y métricas de monitoreo) y según el método, como el aprendizaje supervisado o no supervisado. También indican qué instrumentación requiere cada técnica. Entre los retos pendientes destacan los falsos positivos y la poca capacidad de explicar las anomalías detectadas.

Dato concreto: Los autores distinguen entre *fallo*, que es la incapacidad real de un servicio para cumplir su función, y *anomalía*, que es el síntoma observable de ese fallo, por ejemplo un mayor tiempo de respuesta, una caída del rendimiento o la aparición de eventos de error (Sección 2, *Terminology*).

Nota personal: Esta fuente me aclaró algo que antes confundía: lo que se observa en el monitoreo no siempre es el problema real, sino una señal de él. En TA-IND-01 la usaré en el apartado de tolerancia a fallos y observabilidad, para analizar si el sistema elegido recoge *logs*, trazas y métricas, y cómo esa información le permite reaccionar cuando uno de sus servicios deja de funcionar.

Ficha 3. Cómo medir la escalabilidad de una aplicación en la nube

Referencia (IEEE): S. Henning y W. Hasselbring, «A configurable method for benchmarking scalability of cloud-native applications,» *Empirical Software Engineering*, vol. 27, n.º 6, 143, nov. de 2022. DOI: 10.1007/s10664-022-10162-1

DOI verificado: 10.1007/s10664-022-10162-1

Resumen: Los autores señalan que la escalabilidad suele presentarse como una ventaja de las aplicaciones nativas de la nube, pero pocas veces se mide de forma comparable. Para resolverlo proponen un método configurable de *benchmarking* con dos métricas: la demanda de recursos, que indica cuántos recursos se necesitan como mínimo para atender cierta carga, y la capacidad de carga, que indica cuánta carga soporta una cantidad dada de recursos. El método se implementa en Theodolite, una herramienta de código abierto que funciona sobre Kubernetes. Lo evaluaron con 24 configuraciones en tres entornos de nube y concluyen que permite medir la escalabilidad en un tiempo razonable.

Dato concreto: El artículo adopta la definición de Herbst et al.: la escalabilidad es la capacidad de un sistema para sostener cargas de trabajo crecientes utilizando recursos adicionales. Además, siguiendo a Weber et al., la precisa con tres atributos: la intensidad de la carga, los objetivos de nivel de servicio (SLO) y los recursos aprovisionados (Sección 2.2.1).

Nota personal: De esta fuente tomaré la definición formal de escalabilidad para el marco teórico de TA-IND-01. Las dos métricas también me dan un criterio para el análisis crítico: si una empresa afirma que su sistema escala, debería poder mostrar cuántos recursos adicionales necesita para atender a más usuarios sin incumplir sus objetivos de servicio.

Ficha 4. Gestión de datos en microservicios

Referencia (IEEE): R. Laigner, Y. Zhou, M. A. V. Salles et al., «Data management in microservices: State of the practice, challenges, and research directions,» *Proceedings of the VLDB Endowment*, vol. 14, n.º 13, págs. 3348-3361, sep. de 2021. DOI: 10.14778/3484224.3484232

DOI verificado: 10.14778/3484224.3484232

Resumen: El estudio examina un tema que a menudo queda en segundo plano en los microservicios: cómo se gestionan los datos cuando cada servicio tiene su propia base de datos. Los autores combinaron tres fuentes de información: la revisión de unos 300 artículos, el análisis de 9 aplicaciones de código abierto y una encuesta en la que participaron más de 120 profesionales. Encontraron que la consistencia entre servicios suele resolverse en el código de la propia aplicación, con mecanismos hechos a medida y comunicación por eventos. Concluyen que faltan sistemas de bases de datos pensados para microservicios y proponen las características que estos deberían ofrecer.

Dato concreto: El 43 % de las respuestas de la encuesta indicó que cada microservicio encapsula su estado en su propio servidor de base de datos administrado (Sección 3.2, p. 3351).

Nota personal: Con este artículo entendí por qué la consistencia de los datos es uno de los puntos más delicados de una arquitectura distribuida: al separar las bases de datos se gana independencia entre servicios, pero se pierden las transacciones sencillas. En TA-IND-01 lo usaré en el análisis de la gestión de datos, para revisar si el sistema elegido emplea una base de datos por servicio, eventos o sagas, y qué problemas de consistencia podría presentar.

Ficha 5. Patrones de despliegue y comunicación

Referencia (IEEE): I. Karabey Aksakalli, T. Çelik, A. B. Can et al., «Deployment and communication patterns in microservice architectures: A systematic literature review,» *Journal of Systems and Software*, vol. 180, 111014, oct. de 2021. DOI: 10.1016/j.jss.2021.111014

DOI verificado: 10.1016/j.jss.2021.111014

Resumen: Una vez que un sistema se divide en microservicios, hay que decidir dónde se ejecutará cada módulo y cómo se comunicarán entre ellos. Este artículo aborda esas dos decisiones mediante una revisión sistemática de la literatura: de 239 artículos revisados, los autores seleccionaron 38 estudios primarios. Identificaron tres enfoques de despliegue y siete patrones de comunicación, entre ellos la comunicación síncrona, la asíncrona, la de publicación/suscripción y la basada en middleware de mensajería. También describen ocho desafíos de despliegue y seis de comunicación. Su conclusión es que estas decisiones pesan tanto como la identificación de los servicios en el éxito de la arquitectura.

Dato concreto: Los tres enfoques de despliegue identificados son el despliegue *serverless*, una instancia de servicio por máquina virtual y una instancia de servicio por contenedor (Sección 4.4.1, subsecciones 4.4.1.1 a 4.4.1.3).

Nota personal: Esta revisión me dio un vocabulario ordenado para describir la infraestructura de un sistema distribuido. En TA-IND-01 la usaré en la sección de descripción de la arquitectura, para clasificar cómo se despliega el sistema elegido (contenedores, máquinas virtuales o *serverless*) y qué tipo de comunicación usan sus servicios, y después relacionar esas decisiones con sus ventajas y limitaciones.

Declaración de uso de inteligencia artificial

Desarrollé esta actividad en un trabajo conjunto con Claude (Anthropic). Mi parte fue definir los criterios de búsqueda y los requisitos a partir de las instrucciones de la asignatura, dirigir el desarrollo, revisar las versiones del documento y pedir las correcciones necesarias, por ejemplo en el formato de los enlaces. Claude buscó y preseleccionó las fuentes, consultó sus metadatos en Crossref (autores, año, título, revista, volumen, número y páginas o número de artículo) y comprobó que cada DOI correspondiera a la obra citada. También localizó los datos concretos en el texto completo de cada artículo, propuso la redacción de los resúmenes y de las notas personales, y preparó los archivos LaTeX, BibLaTeX y README del repositorio.

Las respuestas de la herramienta no se usan como fuente: cada afirmación remite a uno de los cinco artículos. Para la Ficha 1, las secciones se consultaron en el manuscrito aceptado de acceso abierto publicado por los autores, y para la Ficha 2, en la versión de acceso abierto del artículo en arXiv (2105.12378); las demás se consultaron en la versión del editor. Revisé el documento final antes de entregarlo.

Referencias

[1] H. Rodrigues, A. Rito Silva y A. Avritzer, «Assessment of performance and its scalability in microservice architectures: Systematic literature review,» *Journal of Systems and Software*, vol. 230, 112500, dic. de 2025. DOI: 10.1016/j.jss.2025.112500.

[2] J. Soldani y A. Brogi, «Anomaly Detection and Failure Root Cause Analysis in (Micro) Service-Based Cloud Applications: A Survey,» *ACM Computing Surveys*, vol. 55, n.º 3, págs. 1-39, mar. de 2023. DOI: 10.1145/3501297.

[3] S. Henning y W. Hasselbring, «A configurable method for benchmarking scalability of cloud-native applications,» *Empirical Software Engineering*, vol. 27, n.º 6, 143, nov. de 2022. DOI: 10.1007/s10664-022-10162-1.

[4] R. Laigner, Y. Zhou, M. A. V. Salles, Y. Liu y M. Kalinowski, «Data management in microservices: State of the practice, challenges, and research directions,» *Proceedings of the VLDB Endowment*, vol. 14, n.º 13, págs. 3348-3361, sep. de 2021. DOI: 10.14778/3484224.3484232.

[5] I. Karabey Aksakalli, T. Çelik, A. B. Can y B. Tekinerdoğan, «Deployment and communication patterns in microservice architectures: A systematic literature review,» *Journal of Systems and Software*, vol. 180, 111014, oct. de 2021. DOI: 10.1016/j.jss.2021.111014.